Software Design Description

Looper Application — Built on miniaudioengine SDK

Aidan Clyens

Contents

1. Introduction **3**

2. Stakeholders and Design Concerns **4**
2.1. Stakeholders 4
2.2. Design Concerns 5

3. Design Views **6**
3.1. Context View 6
3.2. Composition View 7
3.2.1. Looper Application Components 7
3.2.2. Loop Collection 8
3.2.3. SDK Track Hierarchy (runtime) 8
3.3. Logical View 9
3.3.1. Metronome Flow 9
3.3.2. Record Loop Flow 10
3.3.3. Overdub Flow 11
3.3.4. Delete Loop Flow 12
3.3.5. Loop State Machine 13
3.4. Dependency View 14
3.5. Information View 15
3.5.1. Metronome 15
3.5.2. Loop 16
3.5.3. LoopEngine 17
3.5.4. LooperSession 18
3.6. Interface View 19
3.6.1. User-Facing Operations 19
3.6.2. SDK API Touchpoints 20
3.7. Interaction View 21
3.7.1. Session Initialisation 21
3.7.2. First Loop Recording and Playback 22
3.7.3. Overdub (Second Loop) 23
3.7.4. Delete a Loop 24
3.8. Structure View 25
3.8.1. Project Structure 25
3.8.2. Thread Structure 26
3.8.3. Real-Time Safety Boundary 28

4. Design Rationale **29**
4.1. Why a Separate Application Layer? 29
4.2. Quantisation to Bar Boundaries 29
4.3. In-Memory Capture Buffer 29
4.4. Mute via SDK `enable_output` 29
4.5. Metronome Thread vs. Real-Time Callback 30
4.6. Single Audio Input Device 30

1. Introduction

This document describes the software design for a **looper application** built on the miniaudioengine SDK. The looper targets embedded Linux platforms (e.g. Raspberry Pi) connected to a USB or I²S audio interface. It allows a user to record repeating audio loops quantised to a metronome beat, layer multiple loops on top of each other across independent tracks, and route the combined mix to the audio interface output.

The design reuses the SDK's `TrackService`, `DeviceService`, and `Track` hierarchy without modification. All looper-specific logic — timing, state management, and recording — lives in a new application layer above the SDK.

2. Stakeholders and Design Concerns

2.1. Stakeholders

Stakeholder	Responsibilities	Design Concerns
End User (Musician)	- Set metronome BPM. - Record audio loops from an audio interface. - Overdub additional loops on top of existing recordings. - Mute / unmute individual loops. - Delete individual loops or all loops. - Monitor combined playback through the audio interface output.	DC-L01 DC-L02 DC-L03 DC-L04 DC-L05 DC-L06 DC-L07
Embedded Platform (Raspberry Pi)	- Run real-time audio processing under Linux. - Provide stable, low-latency audio I/O via an attached audio interface.	DC-L08 DC-L09
miniaudioengine SDK	- Supply audio device enumeration, track management, and real-time audio routing primitives.	DC-L09 DC-L10

2.2. Design Concerns

ID	Description	Relevant Views
DC-L01	User can configure a metronome BPM.	Logical, Information
DC-L02	User can start and stop the metronome.	Logical, Interaction
DC-L03	User can record a loop from the audio interface input, quantised to the metronome bar boundary.	Logical, Interaction
DC-L04	User can play back a recorded loop in a repeating cycle.	Logical, Interaction
DC-L05	User can overdub additional loops by recording on additional tracks simultaneously with existing playback.	Logical, Interaction
DC-L06	User can mute or unmute any individual loop.	Logical, Information
DC-L07	User can delete an individual loop or clear all loops.	Logical, Interaction
DC-L08	Looper runs on Linux (Raspberry Pi); audio I/O via an audio interface device.	Context
DC-L09	Looper integrates the miniaudioengine SDK for all audio device access, track management, and real-time mixing.	Context, Dependency
DC-L10	Loop boundaries are always aligned to a bar-length time window determined by BPM and beats-per-bar.	Logical, Interaction

3. Design Views

3.1. Context View

Describes the looper application in context with its external environment.

Design Concern	
DC-L08	Looper runs on Linux (Raspberry Pi); audio I/O via an audio interface device.
DC-L09	Looper integrates the miniaudioengine SDK for all audio device access and routing.

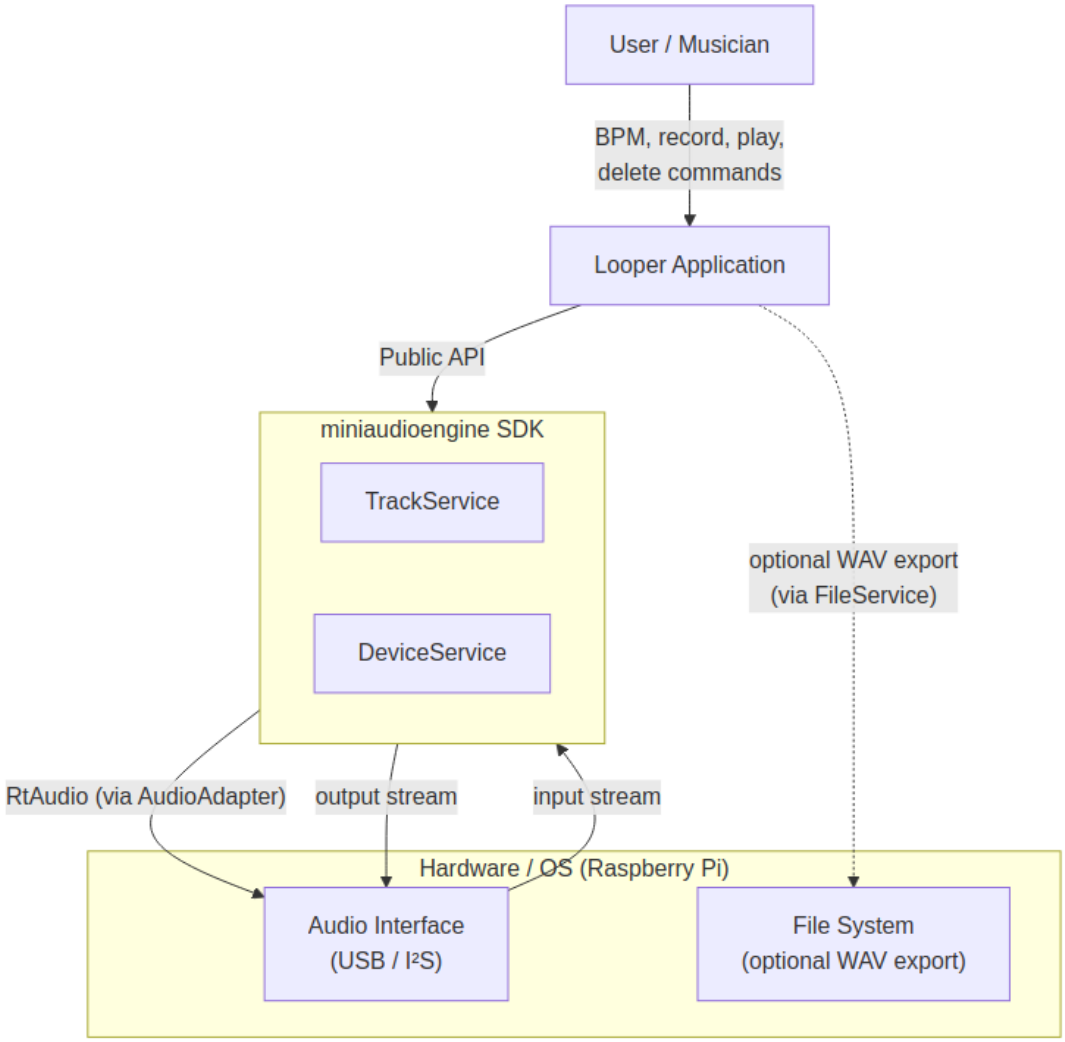

Figure 1: Loop Context View

3.2. Composition View

3.2.1. Looper Application Components

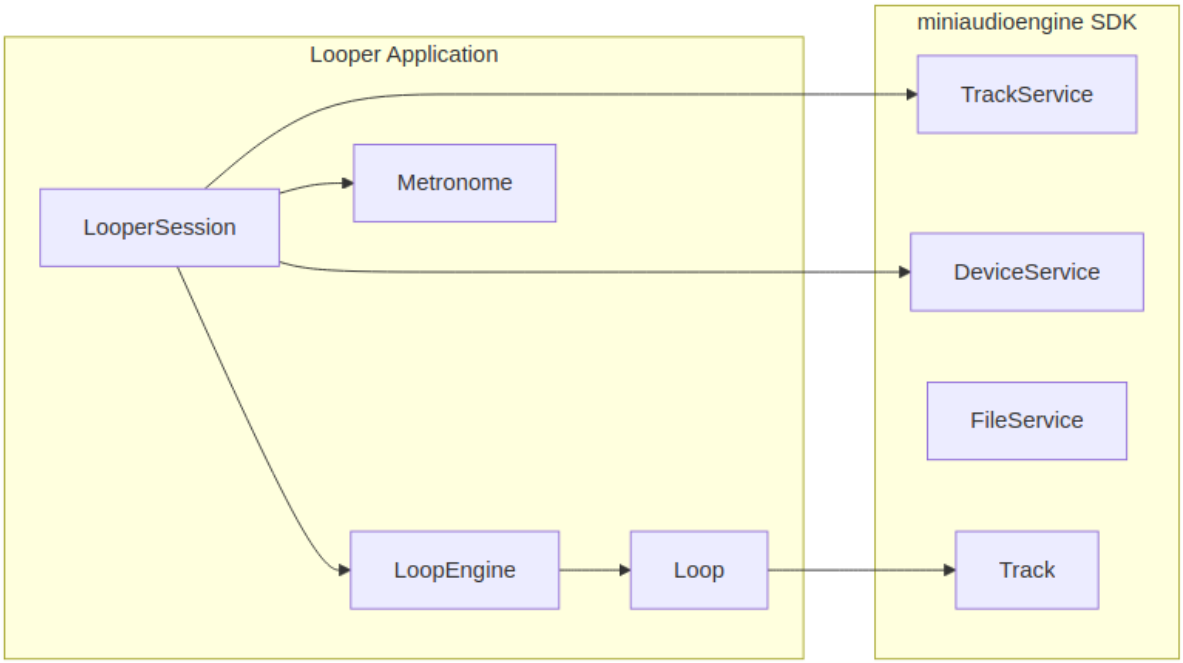

Figure 2: Looper Application Components View

3.2.2. Loop Collection

Each active loop maps to one child `Track` under the SDK's `MainTrack`.

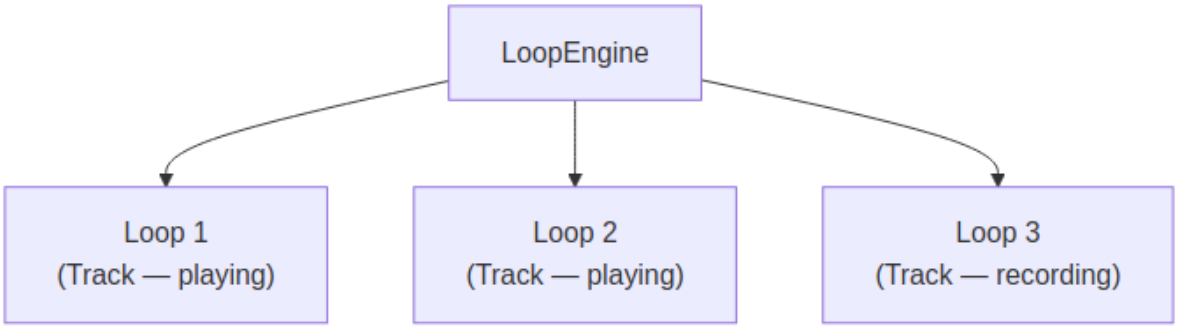

Figure 3: Loop Collection View

3.2.3. SDK Track Hierarchy (runtime)

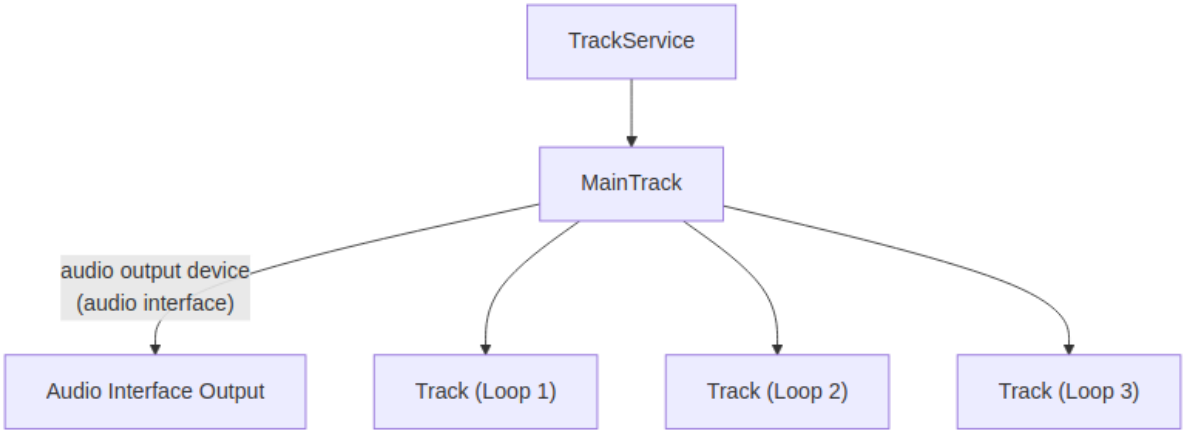

Figure 4: SDK Track Hierarchy View

3.3. Logical View

3.3.1. Metronome Flow

Design Concern	
DC-L01	User can configure a metronome BPM.
DC-L02	User can start and stop the metronome.

The `Metronome` tracks the current beat and bar position in time. It fires a `on_bar_start` callback at the beginning of every bar. The loop length is derived from BPM and beats-per-bar.

```
loop_duration_s = (60.0 / BPM) × beats_per_bar × bars_per_loop
```

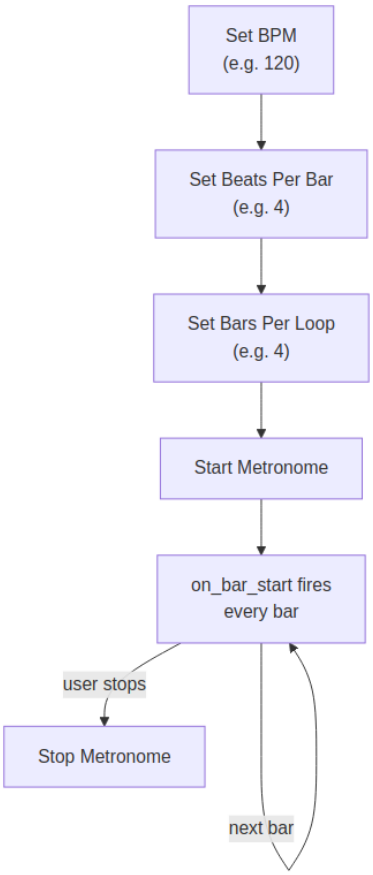

Figure 5: Metronome Flow

3.3.2. Record Loop Flow

Design Concern	
DC-L03	User can record a loop from the audio interface input, quantised to the metronome bar boundary.
DC-L10	Loop boundaries align to the bar-length window.

Recording does not begin immediately when the user presses record. Instead the `LoopEngine` arms the next loop and begins capture precisely when the next bar starts.

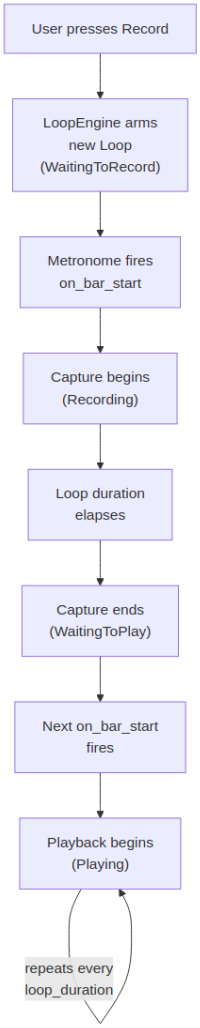

Figure 6: Record Loop Flow

3.3.3. Overdub Flow

Design Concern	
DC-L05	User can overdub additional loops on top of existing playback.

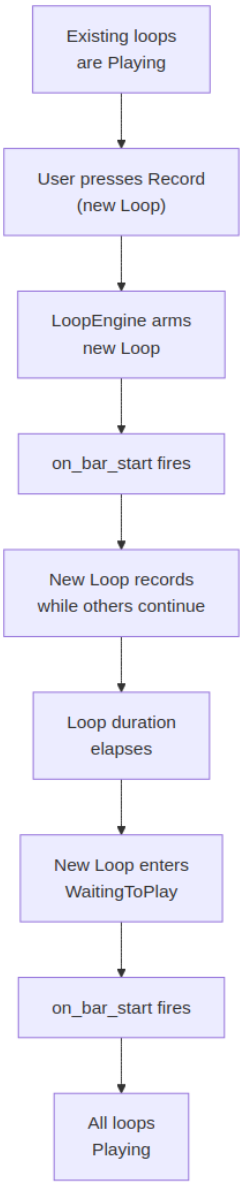

Figure 7: Overdub Flow

3.3.4. Delete Loop Flow

Design Concern	
DC-L07	User can delete an individual loop or clear all loops.

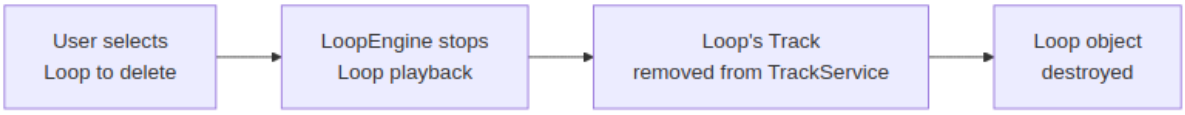

Figure 8: Diagram 8

3.3.5. Loop State Machine

Each `Loop` has an independent state machine.

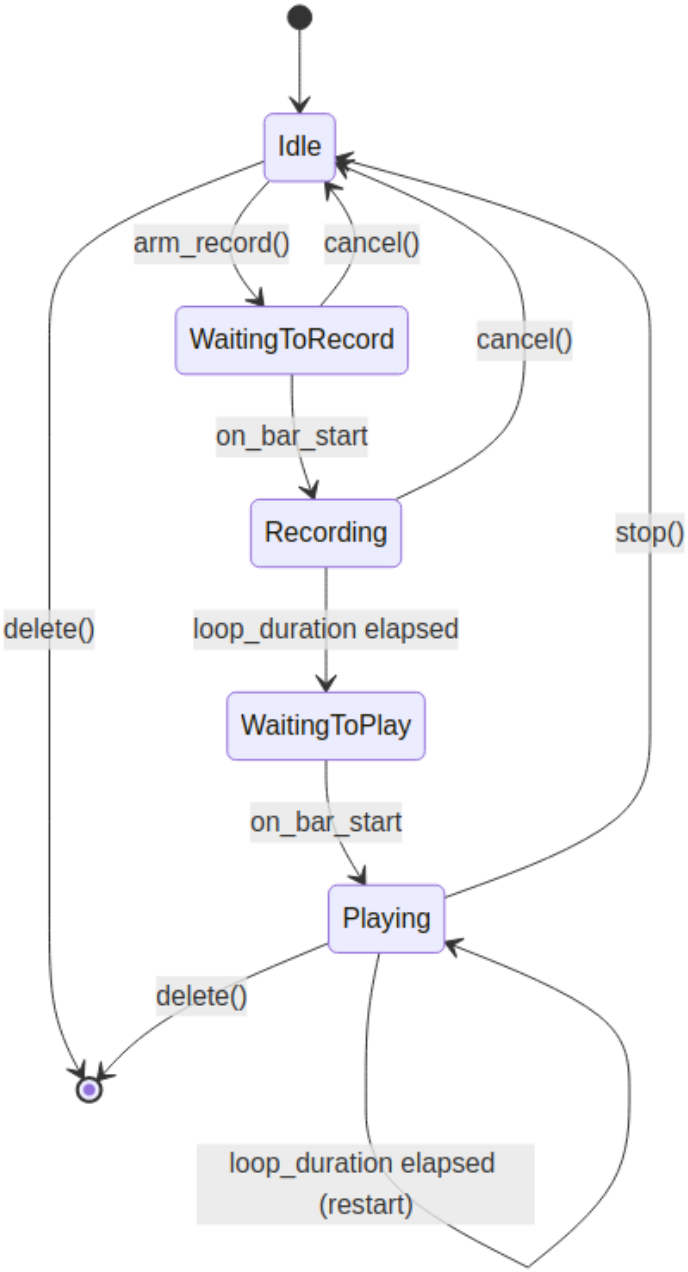

Figure 9: Diagram 9

3.4. Dependency View

The looper application sits above the miniaudioengine SDK. It does not depend on any SDK-internal layer; all access is through the SDK's public header files in `include/miniaudioengine/`.

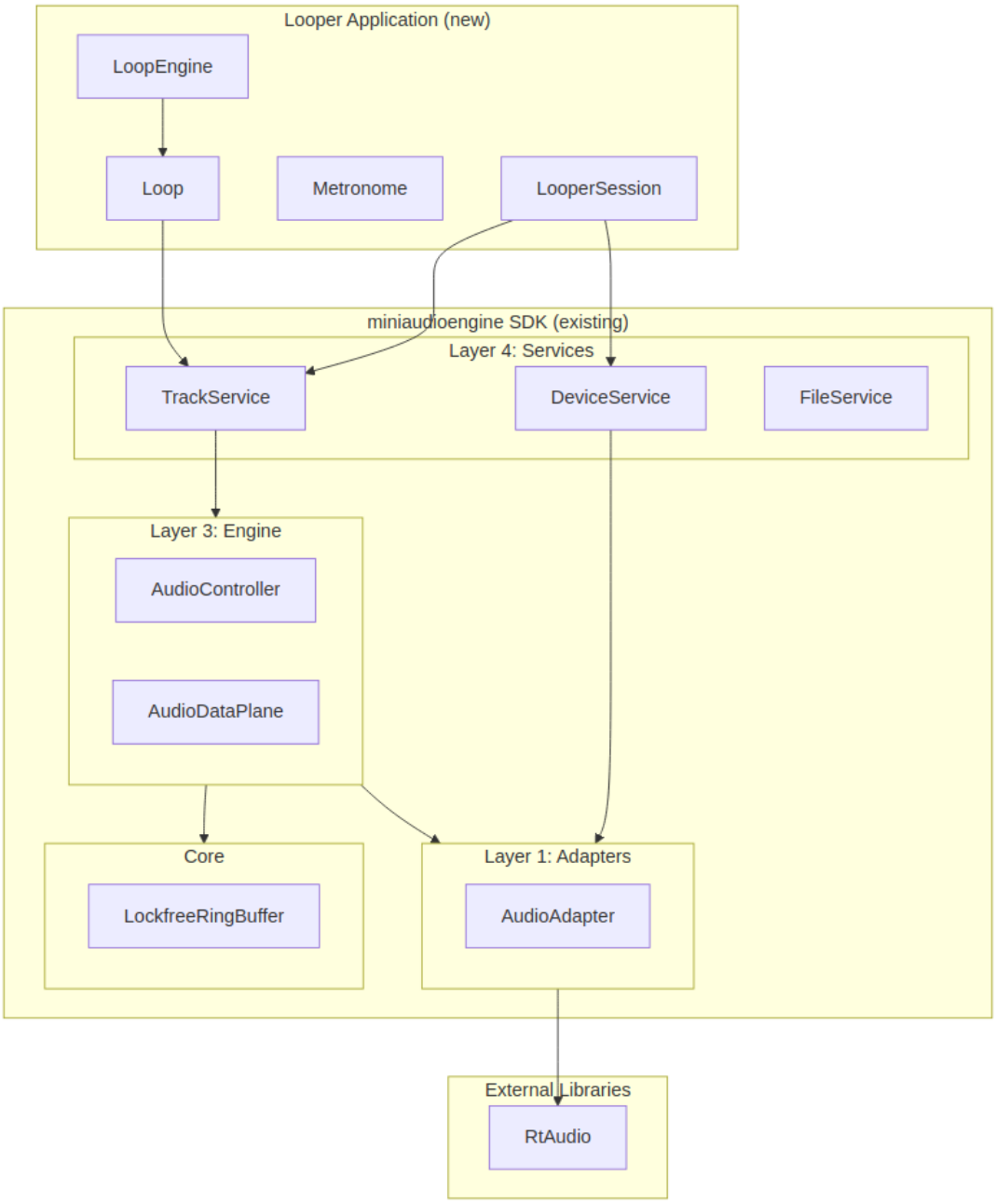

Figure 10: Diagram 10

3.5. Information View

3.5.1. Metronome

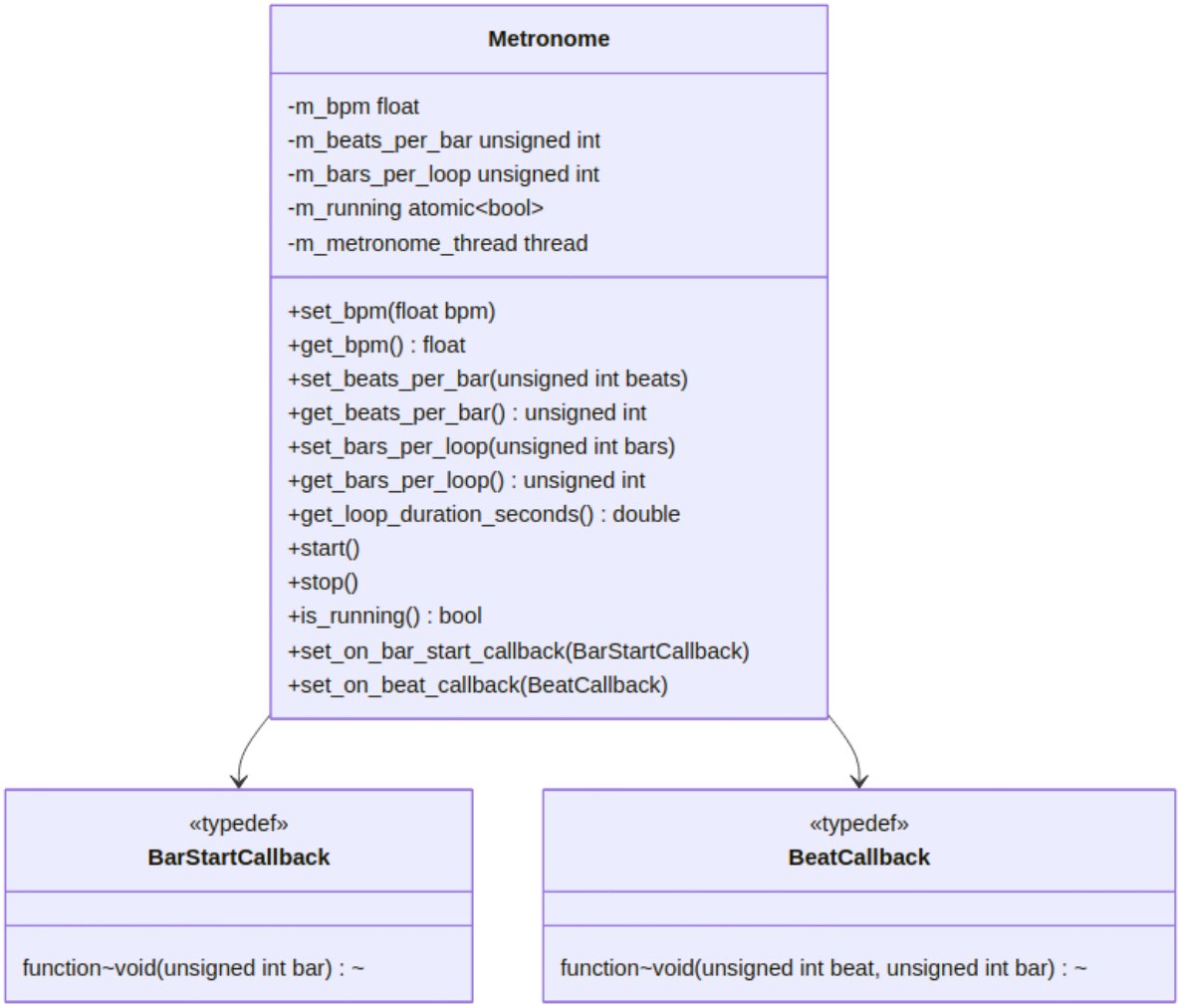

Figure 11: Diagram 11

3.5.2. Loop

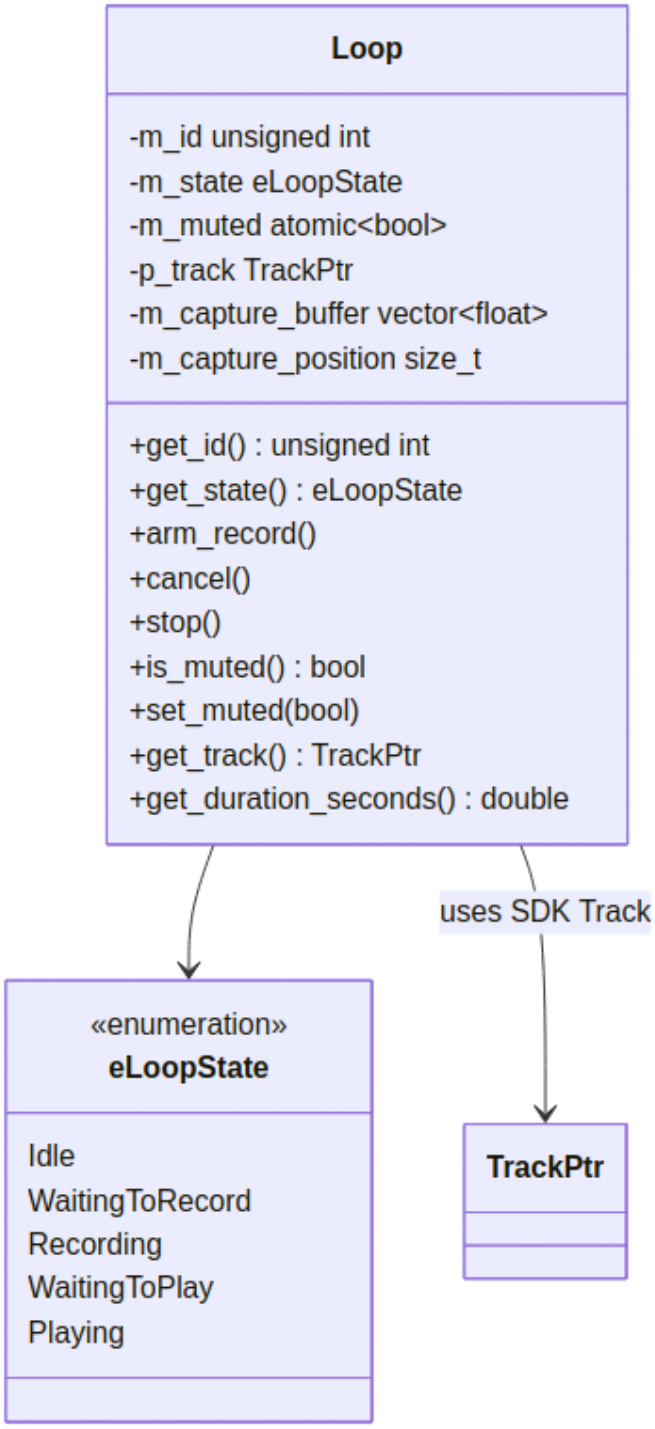

Figure 12: Diagram 12

3.5.3. LoopEngine

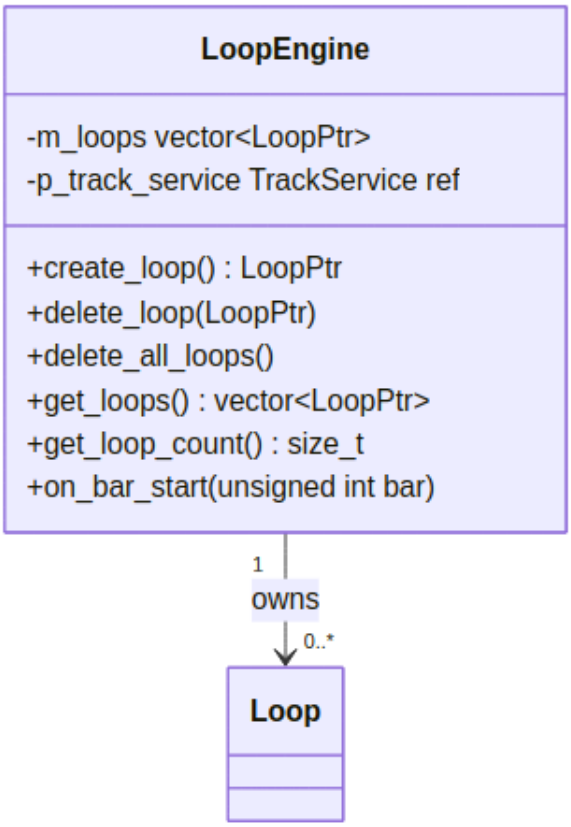

Figure 13: Loop Engine Class Diagram

3.5.4. LooperSession

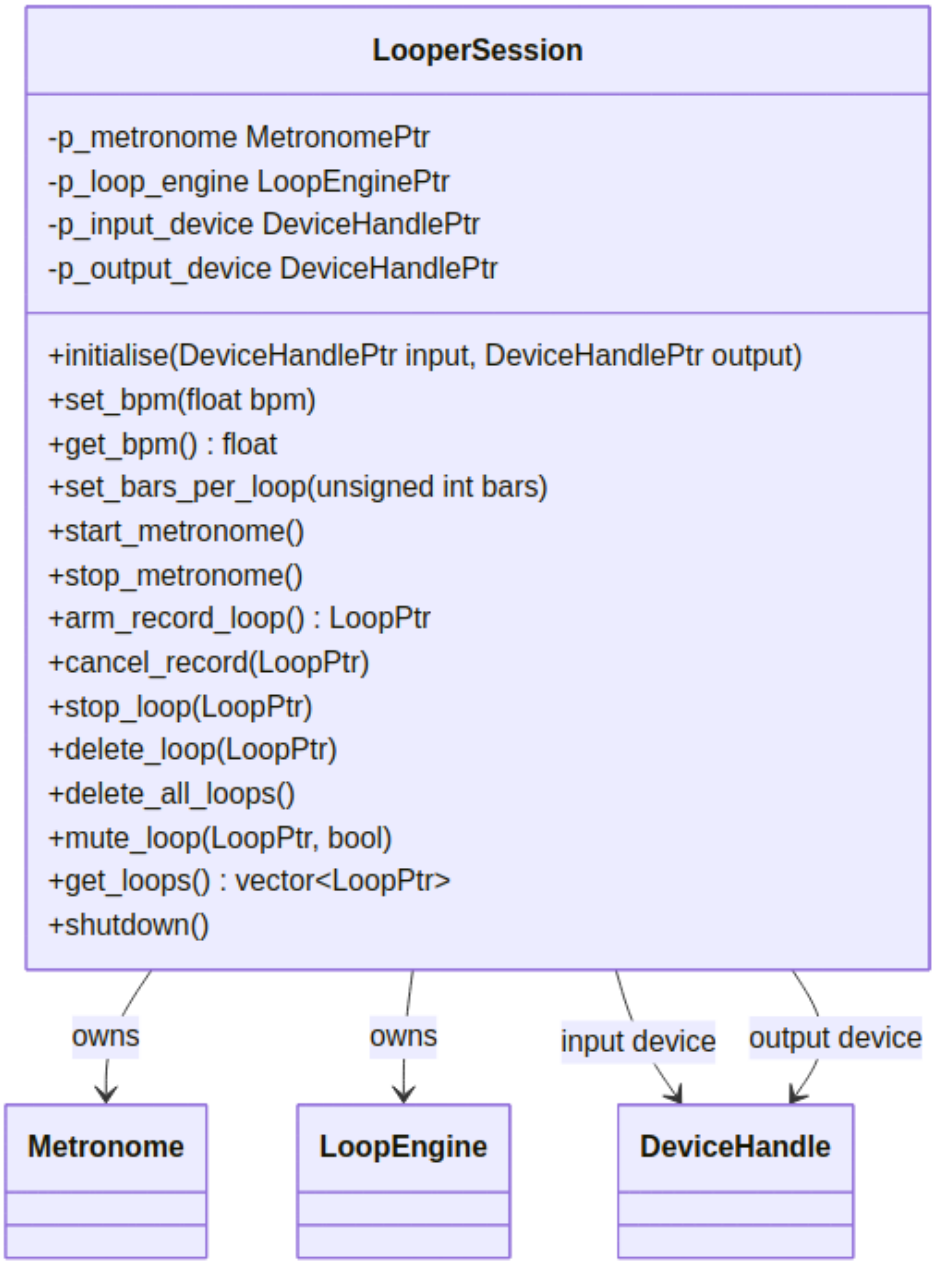

Figure 14: LooperSession Class Diagram

3.6. Interface View

3.6.1. User-Facing Operations

Operation	Method	Notes
Set BPM	`LooperSession::set_bpm(float)`	Any time before or after metronome starts; takes effect on next bar.
Set loop length	`LooperSession::set_bars_per_loop(unsigned int)`	Default 4 bars.
Start metronome	`LooperSession::start_metronome()`	Launches metronome thread; begins firing bar-start callbacks.
Stop metronome	`LooperSession::stop_metronome()`	Stops timing; in-progress recording is cancelled.
Arm loop record	`LooperSession::arm_record_loop()`	Returns a `LoopPtr`; capture begins at next bar boundary.
Stop loop	`LooperSession::stop_loop(LoopPtr)`	Halts playback; loop enters `Idle`.
Mute loop	`LooperSession::mute_loop(LoopPtr, bool)`	Toggles `Track::enable_output(bool)` on the underlying SDK track.
Delete loop	`LooperSession::delete_loop(LoopPtr)`	Removes the loop and its SDK track.
Delete all	`LooperSession::delete_all_loops()`	Removes all loops and tracks.

3.6.2. SDK API Touchpoints

Looper Component	SDK Call	Purpose
`LooperSession::initialize()`	`DeviceService::get_audio_devices()`	Enumerate audio interface devices.
`LooperSession::initialize()`	`TrackService::set_audio_output_device()`	Route mix to audio interface output.
`LoopEngine::create_loop()`	`TrackService::create_child_track()`	Allocate a new SDK track for the loop.
`LoopEngine::delete_loop()`	`TrackService::remove_track()`	Release the SDK track.
Recording start	`Track::add_audio_input(DeviceHandle, Pan)`	Route audio interface input to loop's track.
Playback start	`Track::add_audio_input(FileHandle, Pan)`	Feed captured audio back via SDK (or in-memory buffer).
Mute	`Track::enable_output(bool)`	Silence a loop without stopping it.
Gain	`Track::set_output_gain(float)`	Per-loop volume control.

3.7. Interaction View

3.7.1. Session Initialisation

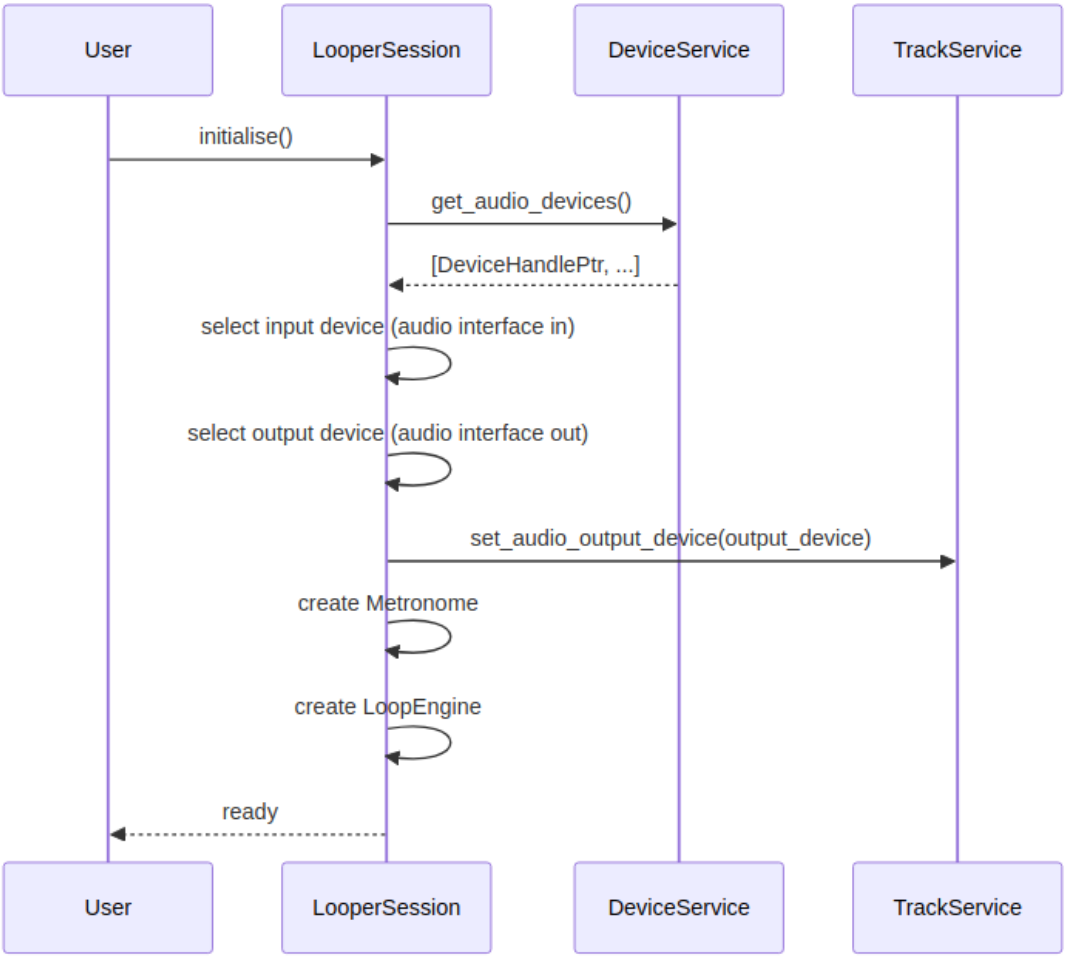

Figure 15: Diagram 15

3.7.2. First Loop Recording and Playback

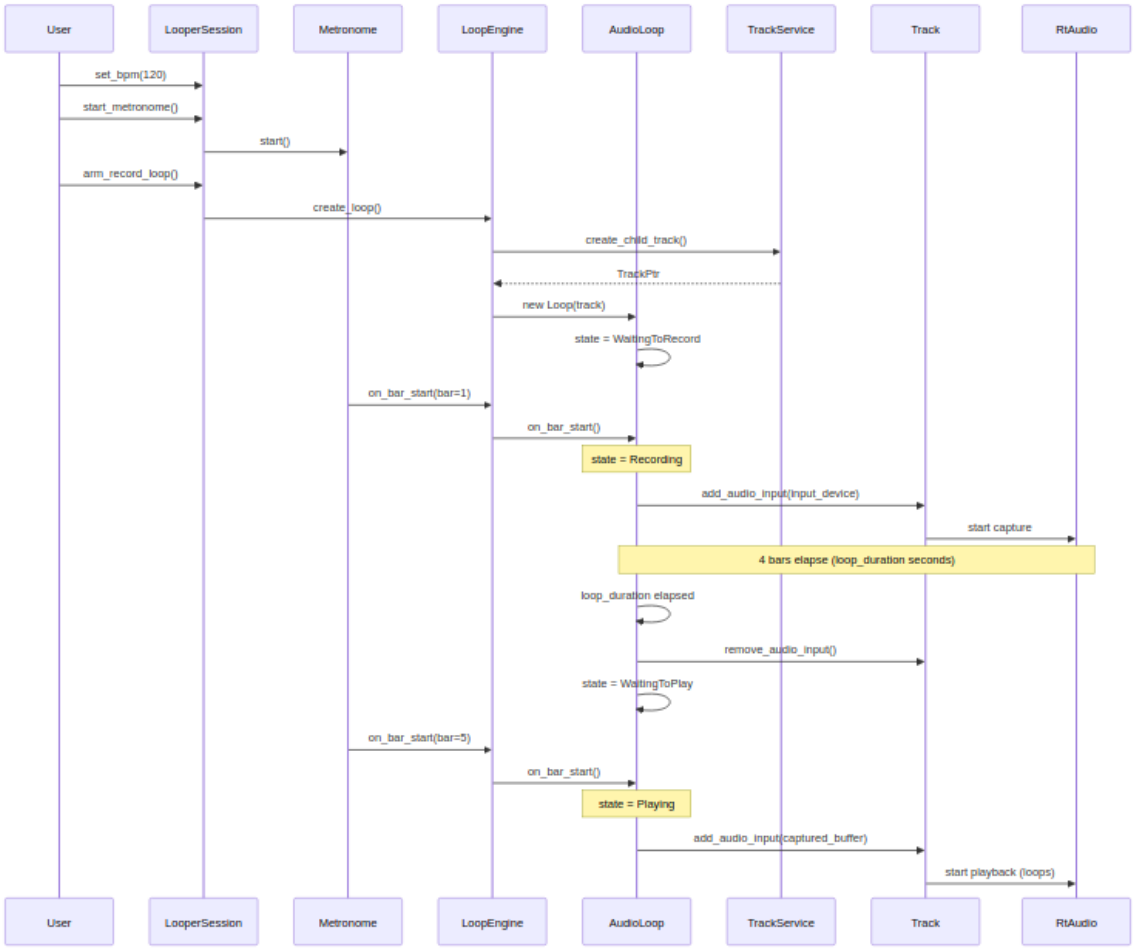

Figure 16: Diagram 16

3.7.3. Overdub (Second Loop)

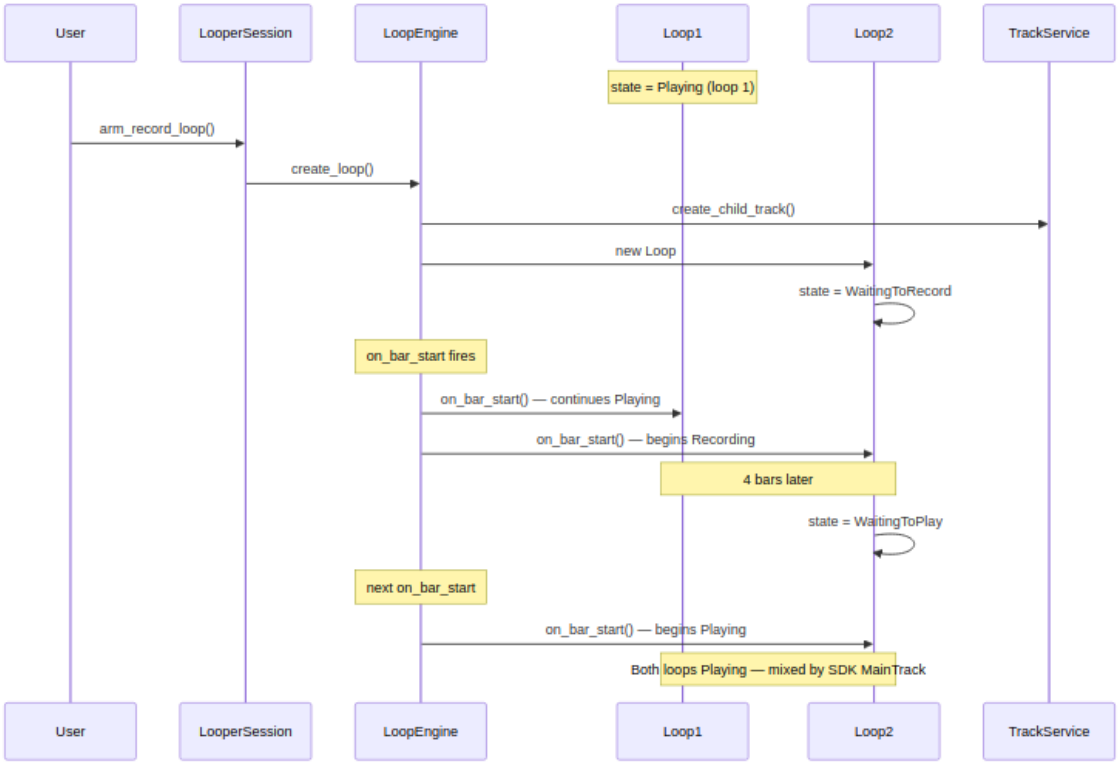

Figure 17: Diagram 17

3.7.4. Delete a Loop

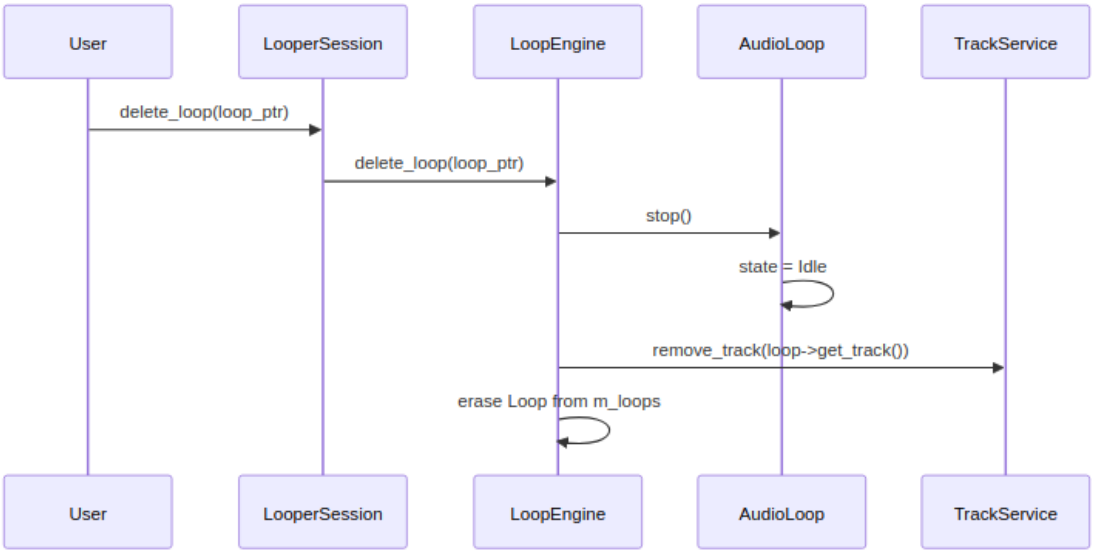

Figure 18: Diagram 18

3.8. Structure View

3.8.1. Project Structure

```
looper/
    CMakeLists.txt
    include/
        metronome.h         # Metronome - timing, BPM, bar callbacks
        loop.h              # Loop - state machine, capture buffer
        loop_engine.h       # LoopEngine - manages collection of Loops
        looper_session.h    # LooperSession - top-level API for the application
    src/
        metronome.cpp
        loop.cpp
        loop_engine.cpp
        looper_session.cpp
    main.cpp                # Entry point - CLI or GPIO input handling
```

The looper links against the installed `miniaudioengine` SDK target from CMake. It does not add any files inside `src/` of the SDK.

3.8.2. Thread Structure

Thread	Owner	Description
Main	OS	User command handling, session control. Non-real-time.
Metronome	`Metronome`	High-priority thread; sleeps with `std::this_thread::sleep_until` to fire bar and beat callbacks.
RtAudio callback	RtAudio (via SDK)	Real-time audio I/O thread; calls SDK `AudioController` internals. No looper logic runs here.

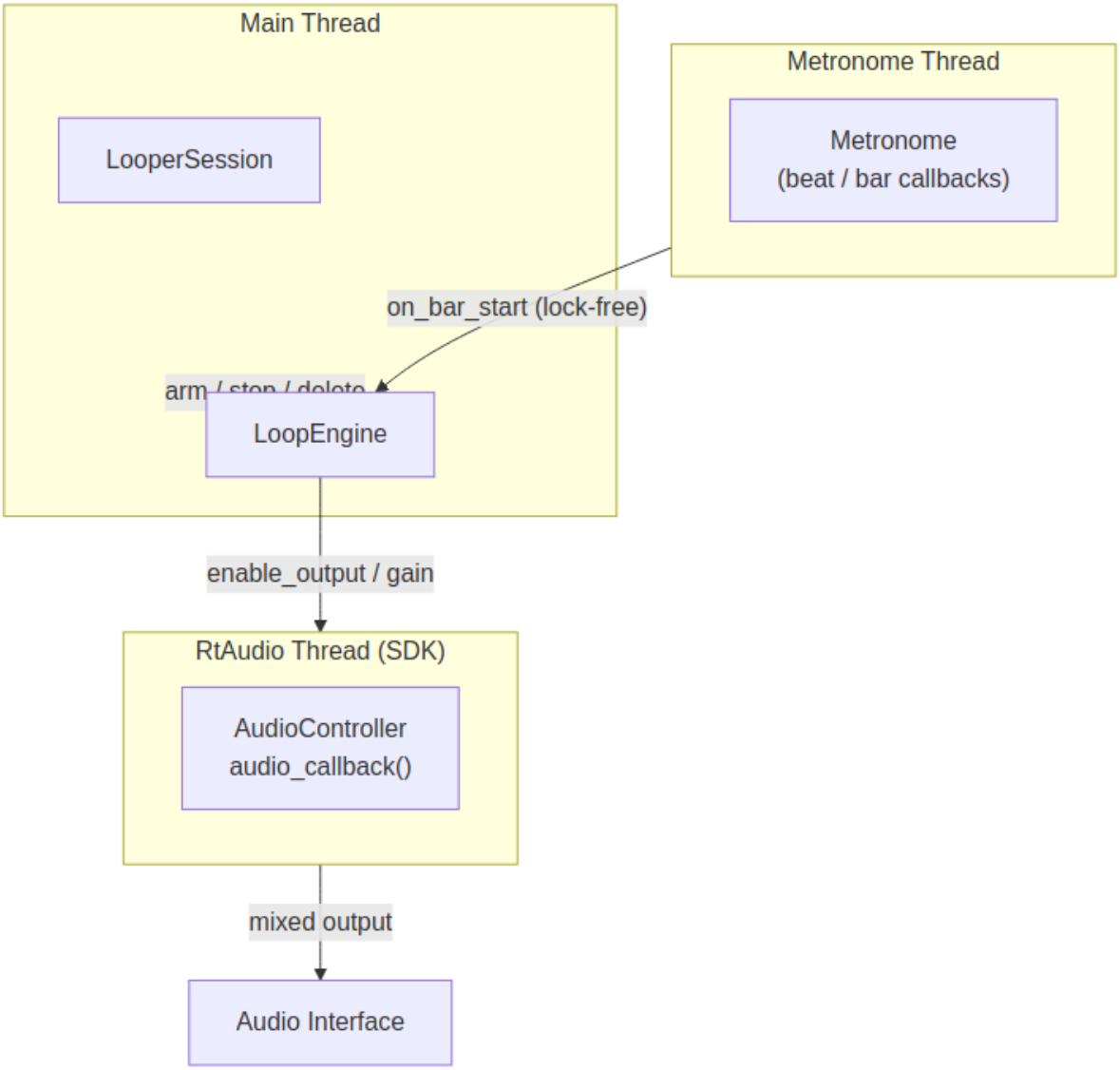

Figure 19: Diagram 19

3.8.3. Real-Time Safety Boundary

The looper application does **not** execute any code directly inside the RtAudio callback. All state transitions (`WaitingToRecord → Recording`, capture buffer management, track enable/disable) happen on the Metronome thread, which is separated from the real-time audio thread. The SDK's `Track::enable_output()` uses `std::atomic` and is therefore safe to call from the Metronome thread concurrently with the RtAudio callback.

4. Design Rationale

4.1. Why a Separate Application Layer?

The looper introduces concepts — metronome-quantised recording, capture buffers, loop state machines — that are not part of the audio engine's responsibilities. Keeping this logic in a dedicated application layer above the SDK preserves the SDK's generality and avoids modifying tested, stable code.

4.2. Quantisation to Bar Boundaries

Recording loops that are not aligned to a bar boundary would create phase drift between layers. By arming the recorder on user input and starting capture only when `on_bar_start` fires, every loop is guaranteed to be exactly `loop_duration_seconds` long, and all loops remain in phase regardless of when the user pressed the button.

4.3. In-Memory Capture Buffer

Rather than writing recordings to a temporary WAV file and reloading them through `FileService`, the `Loop` object owns a `std::vector<float>` capture buffer that is populated sample-by-sample during recording. On transition to `Playing`, the same buffer is fed back into the SDK track as a pre-loaded audio source. This avoids file system latency on the embedded platform and removes the need for temporary file cleanup.

The capture buffer is pre-allocated at `LoopEngine::create_loop()` time (sized to `loop_duration_seconds × sample_rate × channels`) to avoid heap allocation during the recording phase.

4.4. Mute via SDK `enable_output`

Loop muting does not stop the underlying track or reset the playback position — the SDK track continues to process audio internally, but `Track::enable_output(false)` prevents its output buffer from being mixed into the master output. This ensures that un-muting a loop re-enters playback phase-locked with all other active loops.

4.5. Metronome Thread vs. Real-Time Callback

The Metronome runs on a dedicated thread using `std::this_thread::sleep_until` for timing, not inside the RtAudio callback. This means bar-start events are delivered with normal thread-scheduling precision (~1–5 ms jitter on a standard Linux kernel). For a musician-facing looper on a Raspberry Pi this is an acceptable trade-off. If sub-millisecond boundary accuracy is required in future, the bar-start trigger can be moved into the RtAudio callback by tracking elapsed frame counts and comparing against the pre-computed loop-length frame count.

4.6. Single Audio Input Device

The looper uses a single audio input device (the audio interface input) shared by all loops. Recording is serialised: only one loop can be in `Recording` state at any given time. Multiple loops can be simultaneously in `Playing` state; the SDK's `MainTrack` audio callback mixes them.